

Примеры инфраструктурных решений, применяющихся в крупных сетевых проектах

1. Пример реализации инфраструктуры в Google

Google — огромная сервисно-ориентированная платформа для построения масштабируемых приложений, позволяющая выпускать и поддерживать множество конкурентоспособных интернет-приложений, работающих на уровне всей глобальной сети.

Система включает в себя около 1 миллиона недорогих серверов (почти 2 % всех серверов в мире).

Google включает в себя более 400 GFS кластеров. Один кластер может состоять из 1000 или даже 5000 компьютеров.

Десятки и сотни тысяч компьютеров получают данные из GFS кластеров, которые насчитывают более 10 петабайт дискового пространства. Суммарные пропускная способность операций записи и чтения между дата центрами может достигать 40 гигабайт в секунду

Система BigTable позволяет хранить миллиарды ссылок (URL), сотни терабайт снимков со спутников, а также настройки миллионов пользователей Google визуализирует свою инфраструктуру в виде трехслойного стека:

- Продукты: поиск, реклама, электронная почта, карты, видео, чат, блоги и т.п.
- Распределенная инфраструктура системы: GFS, MapReduce и BigTable
- Вычислительные платформы: множество компьютеров во множестве дата-центров

При этом поддерживаются два принципа:

- Легкое развертывание для компании при низком уровне издержек
- Больше денег вкладывается в оборудование для исключения возможности потерь данных

Распределенная файловая система GFS (Google File System)

GFS является основной платформой хранения информации в Google.

GFS — большая распределенная файловая система, способная хранить и обрабатывать огромные объемы информации.

Файлы в GFS организованы иерархически, при помощи каталогов, как и в любой другой файловой системе, и идентифицируются своим путем. С файлами в GFS можно выполнять обычные операции: создание, удаление, открытие, закрытие, чтение и запись.

GFS поддерживает резервные копии, или снимки (snapshot). Можно создавать такие резервные копии для файлов или дерева директорий, причем с небольшими затратами.

Архитектура GFS

В системе существуют мастер-сервера и чанк-сервера, собственно, хранящие данные.

Как правило, GFS-кластер состоит из одной главной машины мастера (master) и множества машин, хранящих фрагменты файлов чанк-серверы (chunkservers). Клиенты имеют доступ ко всем этим машинам. Файлы в GFS разбиваются на куски — чанки (chunk, можно сказать фрагмент). Чанк имеет фиксированный размер, который может настраиваться. Каждый такой чанк имеет уникальный и глобальный 64-битный ключ, который выдается мастером при создании чанка. Чанк-серверы хранят чанки, как обычные Linux файлы, на локальном жестком диске. Для надежности каждый чанк может реплицироваться на другие чанк-серверы. Обычно используются три реплики.

Мастер отвечает за работу с метаданными всей файловой системы. Мастер хранит три важных вида метаданных: пространства имен файлов и чанков, отображение файла в чанки и положение реплик чанков. Все метаданные хранятся в памяти мастера.

Также мастер контролирует всю глобальную деятельность системы такую, как управление свободными чанками, сборка мусора (сбор более ненужных чанков) и перемещение чанков между чанк-серверами. Мастер постоянно обменивается сообщениями (HeartBeat messages) с чанк-серверами, чтобы отдать инструкции, и определить их состояние.

Клиент взаимодействует с мастером только для выполнения операций, связанных с метаданными. Все операции с самими данными производятся напрямую с чанк-серверами.

Разработчики не используют кеширование данных, правда, клиенты кешируют метаданные. На чанк-серверах операционная система Linux и так кеширует наиболее используемые блоки в памяти.

Использование одного мастера существенно упрощает архитектуру системы. Позволяет производить сложные перемещения чанков, организовывать репликации, используя глобальные данные.

Важная часть метаданных — это лог операций. Мастер хранит последовательность операций критических изменений метаданных. По этим отметкам в логе операций, определяется логическое время системы.

Лог операций реплицируется на несколько удаленных машин, и система реагирует на клиентскую операцию, только после сохранения этого лога на диск мастера и диски удаленных машин.

Мастер восстанавливает состояние системы, исполняя лог операций. Лог операций сохраняет относительно небольшой размер, сохраняя только последние операции. В процессе работы мастер создает контрольные точки, когда размер лога превосходит некоторой величины, и восстановить систему можно только до ближайшей контрольной точки. Далее по логу можно заново воспроизвести некоторые операции, таким образом, система может откатываться до точки, которая находится между последней контрольной точкой и текущим временем.

Взаимодействия внутри системы

Каждое изменение чанка должно дублироваться на всех репликах и изменять метаданные. В GFS мастер дает чанк во владение(lease) одному из серверов, хранящих этот чанк. Такой сервер называется первичной (primary) репликой. Остальные реплики объявляются вторичными (secondary). Первичная реплика собирает последовательные изменения чанка, и все реплики следуют этой последовательности, когда эти изменения происходят.

GFS поддерживает такую операцию, как атомарное добавление данных в файл.

Резервные копии (мгновенный снимок) файла или дерева директорий, которые создаются почти мгновенно, при этом, почти не прерывая выполняющиеся операции в системе. Это получается за счет технологии похожей на copy on write.

Операции, выполняемые мастером

Мастер важное звено в системе. Он управляет репликациями чанков: принимает решения о размещении, создает новые чанки, а также координирует различную деятельность внутри системы для сохранения чанков полностью реплицированными, балансировки нагрузки на чанк-серверы и сборки неиспользуемых ресурсов.

В отличие от большинства файловых систем GFS не хранит состав файлов в директории. GFS логически представляет пространство имен, как таблицу, которая отображает каждый путь в метаданные.

Каждая операция мастера требует установления некоторых блокировок. В этом месте в системе используются блокировки чтения-записи.

Кластеры GFS, являются сильно распределенными и многоуровневыми. Обычно, такой кластер имеет сотни чанк-серверов, расположенных на разных стойках. Многоуровневое распределение представляет очень сложную задачу надежного, масштабируемого и доступного распространения данных.

Политика расположения реплик старается удовлетворить следующим свойствам: максимизация надежности и доступности данных и максимизация использование сетевой пропускной способности. Реплики должны быть расположены не только на разных дисках или разных машинах, но и более того на разных стойках.

Когда мастер создает чанк, он выбирает где разместить реплику. Он исходит из нескольких факторов:

- Желательно поместить новую реплику на чанк-сервер с наименьшей средней загруженностью дисков.
- Желательно ограничить число новых создаваемых чанков на каждом чанк-сервере.
- Желательно распределить чанки среди разных стоек.

Как только число реплик падает ниже устанавливаемой пользователем величины, мастер снова реплицирует чанк. Это может случиться по нескольким причинам: чанк-сервер

стал недоступным, один из дисков вышел из строя или увеличена величина, задающая число реплик.

Каждому чанку, который должен реплицироваться, устанавливается приоритет, который тоже зависит от нескольких факторов:

- приоритет выше у того чанка, который имеет наименьшее число реплик.
- чтобы увеличить надежность выполнения приложений, увеличивается приоритет у чанков, которые блокируют прогресс в работе клиента

Мастер выбирает чанк с наибольшим приоритетом и копирует его, отдавая инструкцию одному из чанк-серверов, скопировать его с доступной реплики.

Мастер должен решать, какую из реплик стоит удалить. Как правило, удаляется реплика, которая находится на чанк-сервере с наименьшим свободным местом на жестких дисках.

Мастер собирает мусор. При удалении файла, GFS не требует мгновенного возвращения освободившегося дискового пространства. Он делает это во время регулярной сборки мусора, которая происходит как на уровне чанков, так и на уровне файлов.

При удалении файла приложением, мастер запоминает в логах этот факт, как и многие другие. Тем не менее, вместо требования немедленного восстановления освободившихся ресурсов, файл просто переименовывается, причем в имя файла добавляется время удаления, и он становится невидимым пользователю. мастер, во время регулярного сканирования пространства имен файловой системы, реально удаляет все такие скрытые файлы, которые были удалены пользователем за определенное время. До этого момента файл продолжает находиться в системе, как скрытый, и он может быть прочитан или переименован обратно для восстановления. Когда скрытый файл удаляется мастером, то информация о нем удаляется также из метаданных, а все чанки этого файла отцепляются от него.

Мастер помимо регулярного сканирования пространства имен файлов делает аналогичное сканирование пространства имен чанков. Мастер определяет чанки, которые отсоединены от файла, удаляет их из метаданных и во время регулярных связей с чанк-серверами передает им сигнал о возможности удаления всех реплик, содержащих заданный чанк.

Устойчивость к сбоям и диагностика ошибок

Сбой компонента может быть вызван недоступностью этого компонента или, что хуже, наличием испорченных данных. GFS поддерживает систему в рабочем виде при помощи двух простых стратегий: быстрое восстановление и репликация.

Быстрое восстановление — это, фактически, перезагрузка машины. При этом время запуска очень маленькое, что приводит к маленькой заминке, а затем работа продолжается штатно.

Мастер реплицирует чанк, если одна из реплик стала недоступной, либо повредились данные, содержащие реплику чанка. Поврежденные чанки определяется при помощи вычисления контрольных сумм.

Репликация мастера. Реплицируется лог операций и контрольные точки (checkpoints). Каждое изменение файлов в системе происходит только после записи лога операций на диски мастером, и диски машин, на которые лог реплицируется. Новый мастер является тенью старого, а не точной копией. Поэтому у него есть доступ к файлам только для чтения. То есть он не становится полноценным мастером, а лишь поддерживает лог операций и другие структуры мастера.

Важной частью системы является возможность поддерживать целостность данных. Каждый чанк-сервер должен самостоятельно определять целостность данных. Каждый чанк разбивается на блоки длиной 64 Кбайт. Каждому такому блоку соответствует 32-битная контрольная сумма. Как и другие метаданные эти суммы хранятся в памяти, регулярно сохраняются в лог, отдельно от данных пользователя.

Перед тем как считать данные чанк-сервер проверяет контрольные суммы блоков чанка, которые пересекаются с затребованными данными пользователем или другим чанк-сервером. То есть чанк-сервер не распространяет испорченные данные. В случае несовпадения контрольных сумм, чанк-сервер возвращает ошибку машине, подавшей запрос, и рапортует о ней мастеру.

При добавлении новых данных, верификация контрольных сумм не происходит, а для блоков записывается новые контрольные суммы. В случае если диск испорчен, то это определится при попытке чтения этих данных. При записи чанк-сервер сравнивает только первый и последний блоки, пересекающиеся с границами, в которые происходит запись, поскольку часть данных на этих блоках не перезаписывается и необходимо проверить их целостность.

Организация работы с данными при помощи MapReduce

MapReduce является программной моделью и соответствующей реализацией обработки и генерации больших наборов данных.

Пользователи могут задавать функцию, обрабатывающую пары ключ/значение для генерации промежуточных аналогичных пар, и сокращающую функцию, которая объединяет все промежуточные значения, соответствующие одному и тому же ключу. Программы, написанные в таком функциональном стиле, автоматически распараллеливаются и адаптируются для выполнения на обширных кластерах. Система берет на себя детали разбиения входных данных на части, составления расписания выполнения программ на различных компьютерах, управления ошибками, и организации необходимой коммуникации между компьютерами.

Преимущества использования MapReduce

- Эффективный способ распределения задач между множеством компьютеров
- Обработка сбоев в работе

- Работа с различными типами смежных приложений, таких как поиск или реклама. Возможно предварительное вычисление и обработка данных, подсчет количества слов, сортировка терабайт данных и так далее
- Вычисления автоматически приближаются к источнику ввода-вывода

MapReduce использует три типа серверов:

- Master: назначают задания остальным типам серверов, а также следят за процессом их выполнения
- Map: принимают входные данные от пользователей и обрабатывают их, результаты записываются в промежуточные файлы
- Reduce: принимают промежуточные файлы от Map-серверов и сокращают их указанным выше способом

Технология MapReduce состоит из двух компонентов: соответственно map и reduce. Map отображает один набор данных в другой, создавая тем самым пары ключ/значение, которыми в нашем случае являются слова и их количества. В процессе перемешивания происходит агрегирование типов ключей. Reduction в этом случае просто суммирует все результаты и возвращает финальный результат.

Транспортировка данных между серверами происходит в сжатом виде.

Хранение структурированных данных в BigTable

BigTable является крупномасштабной, устойчивой к потенциальным ошибкам, самоуправляемой системой, которая может включать в себя терабайты памяти и петабайты данных, а также управлять миллионами операций чтения и записи в секунду. BigTable представляет собой распределенный механизм хэширования, построенный поверх GFS. Она предоставляет механизм просмотра данных для получения доступа к структурированным данным по имеющемуся ключу.

С помощью контролирования своих низкоуровневых систем хранения данных, Google получает больше возможностей по управлению и модификации их системой. Подключение и отключение компьютеров к функционирующей системе никак не мешает ей просто работать. Каждый блок данных хранится в ячейке, доступ к которой может быть предоставлен как по ключу строки или столбца, так и по временной метке. Каждая строка может храниться в одной или нескольких таблицах. Таблицы реализуются в виде последовательности блоков по 64 килобайта, организованных в формате данных под названием SSTable.

В BigTable используется три типа серверов:

- Master: распределяют таблицы по Tablet-серверам, а также следят за расположением таблиц и перераспределяют задания в случае необходимости.
- Tablet: обрабатывают запросы чтения/записи для таблиц. Они разделяют таблицы, когда те превышают лимит размера (обычно 100-200 мегабайт). Когда такой сервер прекращает функционирование по каким-либо причинам, 100 других серверов берут на себя по одной таблице и система продолжает работать как будто ничего не произошло.

- Lock: формируют распределенный сервис ограничения одновременного доступа. Операции открытия таблицы для записи, анализа Master-сервером или проверки доступа должны быть взаимоисключающими.

Локальная группировка может быть использована для физического хранения связанных данных вместе, чтобы обеспечить лучшую локализацию ссылок на данные. Таблицы по возможности кэшируются в оперативной памяти серверов.

2. Пример реализации инфраструктуры для проекта Flickr

Flickr является мировым лидером среди сайтов размещения фотографий.

Flickr должны контролировать огромное количество ежесекундно обновляющегося контента, непрерывно пополняющиеся пользователи, постоянный поток новых предоставляемых пользователям возможностей, и при этом поддерживать постоянно высокий уровень производительности.

Статистика:

- Более четырех миллиардов запросов в день
- Примерно 35 миллионов фотографий в кэше прокси-сервера Squid
- Около двух миллионов фотографий в оперативной памяти Squid
- Всего приблизительно 470 миллионов изображений, каждое представлено в 4 или 5 размерах
- 38 тысяч запросов к memcached (12 миллионов объектов)
- 2 петабайта дискового пространства
- Более 400000 фотографий добавляются ежедневно

На проекте Flickr используется практически только свободное программное обеспечение:

- Платформа GNU/Linux (RedHat)
- СУБД MySQL
- Web-сервер Apache
- Скрипты программной логики, написанные на языке PHP и Perl
- Средства сегментирования (Shards)
- Memcached для кэширования часто востребованного контента
- Squid в качестве обратного прокси-сервера для html-страниц и изображений
- Шаблонизатор Smarty
- PEAR для парсинга e-mail и XML
- ImageMagick для обработки изображений
- SystemImager для развертывания элементов конфигурации
- Ganglia для мониторинга распределенных систем
- Subcon для хранения важных системных конфигурационных файлов в SVN-репозитории для легкого развертывания на машины в кластере
- Cvsup для распространения и обновления коллекций файлов по сети.

Входные запросы поступают на сдублированные контроллеры приложений Brocade ServerIron ADX. Они обеспечивают коммутацию приложений и балансировку трафика, основываясь на принципе виртуальных ферм серверов:

- Коммутатор приложений получает все клиентские запросы
- Выбор “лучшего” сервера производится на основании механизма Real-Time Health и наличия требуемой производительности
- Последовательно повышается коэффициент использования для всех серверов
- Интеллектуальное распределение загрузки осуществляется для всех доступных ресурсов
- Метод конфигурируется и выбирается пользователем
- Обеспечивается защита серверной фермы от атак и от неправильной эксплуатации
- Клиенты подсоединяются к серверам приложений используя виртуальный IP (VIP). VIP адреса настраиваются на коммутаторе приложений
- Коммутатор приложений осуществляет трансляцию адресов после выбора нужного сервера, причем сами адреса серверов скрыты.

Центральная база данных включает в себя таблицу пользователей, состоящую из основных ключей пользователей (несколько уникальных идентификационных номеров) и указатель на сегмент, на котором может быть найдена остальная информация о конкретном пользователе.

Все, за исключением фотографий, хранится в базе данных. Для статического контента используются выделенные сервера. Фотографии хранятся в системе хранения данных. После загрузки изображения система выдает различные его размеры, на чем ее работа заканчивается. Метаданные и ссылки на файловые системы, где расположены фотографии, хранятся в базе данных.

В основе масштабируемости лежит репликация. Для поиска по определенной части базы данных создается отдельная копия этого фрагмента. Активная репликация производится по принципу мастер-мастер. Автоматическое инкрементирование идентификационных номеров используется для поддержания системы в режиме одновременной активности обоих серверов в паре.

Каждый сервер в рамках одного сегмента в обычном состоянии нагружен ровно на половину.

Организация резервного копирования данных реализована с помощью процесса `ibbackup`, который выполняется регулярно посредством `cron daemon'a`, причем на каждом сегменте он настроен на разное время. Каждую ночь делается снимок со всего кластера баз данных. Запись или удаление нескольких больших файлов с резервными копиями одновременно на реплицирующую систему хранения может сильно сократить производительность системы в целом на последующие несколько часов из-за процесса репликации. Выполнение этого на активно работающей системе хранения фотографий затруднительно.